

Most Used

GIT COMMANDS

in Software Engineering



X

Revanth Murigipudi

1. Setting up your **GIT** information

Set your name and email so that the commits can associate with your user account

```
git config --global user.name  
"NAME"
```

```
git config --global user.email  
"youremail@abc.com"
```

2. Initialize Repository

Initialize a directory as a git repository
`git init`

To create a local copy of a remote repository (i.e download the repository to your workspace) `git clone`

3. **Add**

Adds the file/files to the git index (called as staging area). Before we make a commit, it's necessary to add the files to the git staging

Adding multiple files

`git add file1 file2 file3`

Adding all files in the current directory

`git add .`

4. Commit

To make a commit. In layman's terms – save the current index (staging) as a snapshot and commit it to project history. It's always advised to provide a proper message along with a commit for easier understanding.

```
git commit --m "your descriptive message  
for the commit"
```

If you want to modify your most recent commit, use the amend command. It lets you modify the log message and also the files in the commit

git commit --amend

If you want to add and commit in one shot, use the -a flag. It automatically stages all the modified and deleted files and commits them as well

git commit --am "your message"

5. Branch

In git, whenever you want to work on something, you always create a branch

You never work on the master branches

To create a new branch

`git branch (branch_name)`

To checkout to the branch

`git checkout (branch_name)`

To create and checkout in one command
`git checkout --b (branch_name)`

Show a list of all branches in your repository
`git branch --list`

Delete local branch
`git branch --d (branch_name)`

Delete remote branch
`git push origin --d (remote_branch_name)`



Git is one of the **most commonly** used **version control systems!**

Similarly, **BossCoder Academy** makes your way to **MAANG** easier and sure shot.

If you aim to get into **MAANG** or similar top product-based companies

Do Checkout 

6. Updates

View files that are staged, not staged, and not being tracked by `git status`

You can set a short alias for your remote repository URL and work with it. You can add as many aliases as you want
`git remote add (alias) (remote URL)`

Push all the local commits in your branch to the remote branch

git push (remote) (branch)

Pull commits from the remote branch into your local branch

git pull

Merge changes/commits from a specified branch into the current branch

git merge (branch_name)

Apply commits from the current working branch in front of the specified branch

git rebase (branch_name)

7. **Stash**

Use the stash command to temporarily save your work without committing.

`git stash`

View all stashes

`git stash list`

Pop the first stash and apply it on the current working copy. The popped stash is removed from the stash list.

`git stash pop`

Apply the first stash on the working copy, but do not remove it from the stash list.

`git stash apply`

Clear the entire stash list

`git stash clear`

8. Messed Up Something? **Try These**

View the entire commit history for the current branch

`git log`

`git log --oneline` (very much readable as it displays only one commit per line in terminal)

Accidentally staged a file? Reset the unstaged file and still retain the changes.

`git reset (file_name)`

View changes between staged files and current working copy.

`git diff`

There are several ways of using the git diff command, but we just showed one most used way above.

Reset everything? THIS IS A DANGEROUS ACTION, use it cautiously. Git will discard all your changes and will exactly make the files as the content in commit specified.

`git reset --hard (commit)`

9. A Little More **Advanced**

Pick a commit from a branch and apply the commit on another

`git cherry --pick (commit_hash)`

Find the commit that caused a bug.

Git uses binary search to accomplish the task

`git bisect (subcommand)`

There are several subcommands like start, bad, good, and depending on the subcommand specified, bisect decides the next operation

Combine multiple commits into one, i.e squash commits. Btw, there's no command like git squash, we use the interactive rebase command to accomplish this

git rebase --i HEAD~N

Squashes the last N commits. You'll have to manually select which commits to squash by mentioning the pick/squash keyword